VALIDATION & DEFENSIVE LOGIC

Block 16 • Trust data only after it earns trust

```
if total_tasks > 0:
    pct = completed_tasks / total_tasks * 100
else:
    print("Total tasks must be greater than zero")
```

Today: stop bad data before it creates bad results.

Where validation fits in the Python story

Before today

- Input arrives as text
- Strings can be normalized
- Numbers can be converted
- Booleans drive decisions

Today

- Check required values
- Check ranges and allowed values
- Check formats
- Explain failures

Why it matters

Bad data is not rare. Real programs must decide whether to accept, warn, or reject it.

Validation is decision logic applied before important work happens.

CONCEPT

What validation means

Validation asks whether data meets the rules required for safe and meaningful processing.

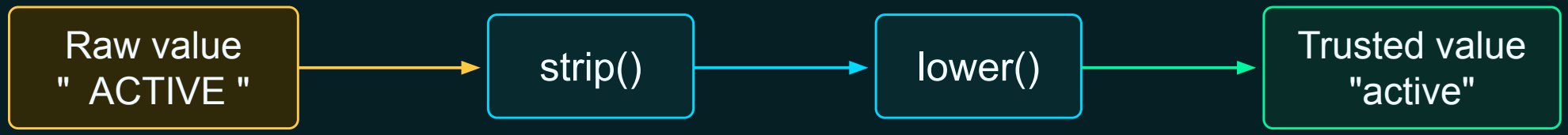
```
record_id = "EQ-1024"

if record_id == "":
    print("Record ID is required")
```

Common validation questions

- Is the value present?
- Is the type usable?
- Is the number in range?
- Is the status allowed?
- Does the identifier format match?
- Do related fields agree?

The program should not assume raw data is correct.

Raw data versus trusted data



Raw values come from users, files, databases, and networks. Trusted values have been cleaned, converted, checked, and intentionally accepted.

Raw is what you received. Trusted is what your program is willing to use.

Blank is not the same as useful

```
record_id = "   "

record_id = record_id.strip()

if record_id == "":
    print("Record ID is required")
```

Why strip first?

A field containing only spaces looks non-empty to Python, but it is not useful to a human system.

value	after strip()	valid?
"EQ-1024"	"EQ-1024"	True
" EQ-1024 "	"EQ-1024"	True
" "	""	False

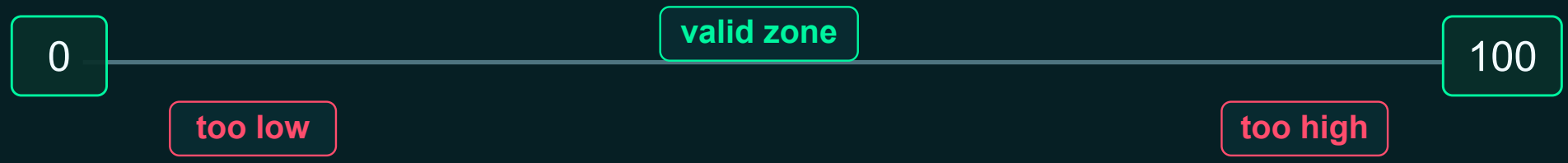
Normalize first, then validate.

Numbers need boundaries

```
completion_percentage = 105

if 0 <= completion_percentage <= 100:
    print("Percentage is valid")
else:
    print("Percentage must be between 0 and 100")
```

Range validation catches nonsense

- Completion cannot be below 0%.
- Completion cannot be above 100%.
- Operating hours cannot be negative.
- Temperature may have an acceptable range.



A number can be numeric and still invalid.

Some fields must be one of a few choices

```
status = "active"

if status == "active" or status == "inactive" or status ==
"retired":
    print("Status is valid")
else:
    print("Unsupported status")
```

Allowed list for today

- active
- inactive
- retired

Later, collections make this cleaner.

input	normalized	accepted?
" Active "	active	True
"retired"	retired	True
"paused"	paused	False
""		False

If only certain words are allowed, say so in code.

Does the value look like the required pattern?

```
equipment_id = "EQ-1045"

has_correct_prefix = equipment_id.startswith("EQ-")
has_expected_length = len(equipment_id) == 7

if has_correct_prefix and has_expected_length:
    print("Identifier format appears valid")
else:
    print("Invalid identifier format")
```

Format rule

Equipment IDs must start with EQ- and contain seven total characters.

EQ-1045 | BAD-1045 | EQ-45

Format validation usually combines several small checks.

Some values are valid only compared with other values

```
completed_tasks = 12
total_tasks = 10

if completed_tasks <= total_tasks:
    print("Task counts are consistent")
else:
    print("Completed tasks cannot exceed total
tasks")
```

Cross-field examples

- completed_tasks <= total_tasks
- end_date after start_date
- amount_spent <= budget
- file_size > 0 when record_count > 0

A field can be valid alone but impossible together with another field.

Validate before calculating

Risky order

```
completion_percentage = completed_tasks / total_tasks * 100

if total_tasks > 0:
    print(completion_percentage)
```

The division happens before the safety check.

Safer order

```
if total_tasks > 0:
    completion_percentage = completed_tasks / total_tasks * 100
    print(completion_percentage)
else:
    print("Total tasks must be greater than zero")
```

Check the denominator before using it.

Defensive programming means preventing obvious failures before they happen.

Valid, warning, or rejected


```
if error_count > 10:
    record_status = "rejected"
elif error_count > 0:
    record_status = "warning"
else:
    record_status = "valid"
```

Not every problem is equally severe.

Humans need the reason, not just True or False

```
if record_id.strip() == "":
    validation_message = "Record ID is missing"
elif not record_id.startswith("EQ-"):
    validation_message = "Record ID must begin with EQ-"
else:
    validation_message = "Record ID is valid"

print(validation_message)
```

Better validation output

- Names the failed rule
- Helps the user fix the data
- Makes debugging faster
- Keeps the program explainable

Computer answer: False → Human answer: Record ID must begin with EQ-

Validation should explain the decision.

DESIGN CHOICE

Fail fast or collect multiple problems?

Fail fast

Stop at the first serious problem.

- Simple if / elif chain
- Good when later checks depend on earlier checks
- Clear single reason

first blocker wins

Collect problems

Check many rules and report all issues.

- More helpful for user cleanup
- Usually needs collections later
- Useful for data-quality reports

all issues listed

Today we focus on readable fail-fast logic, then later improve reporting.

GUIDED EXERCISE

Maintenance record validator

```
record_id = "EQ-1045"
status = "active"
operating_hours = 850
maximum_hours = 1_000
inspection_complete = True
error_count = 2
temperature = 82
```

Your mission

- Create separate Boolean checks.
- Assign valid, warning, or rejected.
- Create a human-readable message.
- Change one value at a time and retest.

Instructor move: have students predict the status before coding. This record has minor errors and temperature outside preferred range.

Do not write one giant condition first. Name the checks.

GUIDED STEP 1

Create named Boolean checks

```
has_record_id = record_id.strip() != ""
has_correct_prefix = record_id.startswith("EQ-")
has_valid_status = status == "active" or status == "inactive"
has_valid_hours = operating_hours >= 0 and maximum_hours > 0
is_within_hour_limit = operating_hours <= maximum_hours
has_valid_temperature = 60 <= temperature <= 80
```

Why names help

- Readable
- Testable
- Printable
- Reusable

```
print(f"Record ID present: {has_record_id}")
print(f"Correct prefix: {has_correct_prefix}")
print(f"Valid status: {has_valid_status}")
```

Intermediate Booleans are your debugging dashboard.

GUIDED STEP 2

Reject blockers first, then warnings

```
if not has_record_id:
    record_status = "rejected"
    status_message = "Record ID is missing"
elif not has_correct_prefix:
    record_status = "rejected"
    status_message = "Record ID format is invalid"
elif not has_valid_status:
    record_status = "rejected"
    status_message = "Status value is invalid"
elif not has_valid_hours:
    record_status = "rejected"
    status_message = "Operating-hour values are invalid"
elif error_count > 5:
    record_status = "rejected"
    status_message = "Too many errors"
```

Why this order?

- Missing required fields stop processing.
- Bad formats stop processing.
- Impossible numeric values stop processing.
- Too many errors stop processing.

Reject checks come before warning checks because they are blockers.

Warnings and success path

```
elif not inspection_complete:
    record_status = "warning"
    status_message = "Inspection is incomplete"
elif not is_within_hour_limit:
    record_status = "warning"
    status_message = "Operating-hour limit exceeded"
elif not has_valid_temperature:
    record_status = "warning"
    status_message = "Temperature outside preferred range"
elif error_count > 0:
    record_status = "warning"
    status_message = "Minor errors require review"
else:
    record_status = "valid"
    status_message = "Record passed validation"
```

Warnings mean

- The record may be usable.
- Someone should review it.
- The message explains what needs attention.
- The else path means every previous problem was ruled out.

The final else is the clean path.

APPLIED CHALLENGE

Build a data intake validator

Record fields

- Record ID
- Source system
- File name and file size
- Record count
- Completion percentage
- Error count
- Approval and security review
- Processing status

Program must produce

- ready, warning, or rejected
- Human-readable explanation
- Ready for processing: True or False
- Formatted summary of the record

```
Status: WARNING
Reason: File contains three noncritical errors.
Ready for processing: False
```

This is the first “realistic data intake” program of the course.

Required validation checks

rule type	check
Required	Record ID is not blank
Format	Record ID uses correct prefix
Allowed value	Source system is recognized
File	File extension is supported
Range	File size and record count are greater than zero
Range	Completion percentage is between 0 and 100
Range	Error count is not negative
Gate	Approval received and security review complete
Gate	Processing status is active

Start by turning each rule into a named Boolean variable.

CHALLENGE ORDER

One possible status priority

Suggested priority: reject invalid or unauthorized records first, warn on noncritical quality issues, and mark ready only after all blockers are cleared.

Order is part of the design. Explain it.

SOLUTION SKELETON

Use named checks, then assign status

```
has_record_id = record_id.strip() != ""
has_ingest_prefix = record_id.startswith("INGEST-")
has_supported_file = file_name.lower().endswith(".csv") or file_name.lower().endswith(".json")
has_positive_size = file_size > 0
has_records = record_count > 0
has_valid_completion = 0 <= completion_percentage <= 100
has_valid_error_count = error_count >= 0

if not has_record_id:
    status = "rejected"
    reason = "Record ID is blank"
elif not has_supported_file:
    status = "rejected"
    reason = "Unsupported file type"
elif error_count > 0:
    status = "warning"
    reason = "File contains noncritical errors"
else:
    status = "ready"
    reason = "Record passed validation"
```

Student reminders

- This is a skeleton, not a complete solution.
- Add the missing rules.
- Print every important check while testing.
- Keep messages useful.

Eight scenarios to prove the logic

scenario	expected
Completely valid record	ready
Blank record ID	rejected
Unsupported file type	rejected
Zero-byte file	rejected
Completion percentage over 100	rejected
Negative error count	rejected
Missing approval	rejected
Valid record with small noncritical errors	warning

A program is not done when it runs once. It is done when expected and actual results match.

KNOWLEDGE CHECK

Can you explain defensive validation?

Ask yourself

- Which values are raw?
- Which values need normalization?
- Which values need conversion?
- Which rules are blockers?
- Which rules are warnings?
- What should the message say?

Leave with this

Validate before you calculate. Normalize before you compare. Explain before you reject.

raw → clean → validate → decide → explain

Next: repeat decisions with loops and collections.